CSC 251 Lecture 11: Panda Series and DataFrames

Pandas

pandas - The main open-source Python library for data analysis for heterogeneous or tabular data.

```
import pandas as pd      or    from pandas import Series, DataFrame
```

The 3 pandas data structures: the Series, DataFrame, and Index.

Pandas Series Objects

Example 1a: Series – a fixed-length ordered dictionary, a mapping of index values to data values (an array plus an index)

```
obj1 = pd.Series([4, 7, -1, 2])
```

```
0    4
1    7
2   -1
3    2
dtype: int64
```

where by default the index (on left) will consist of integers 0 through len – 1.

Example 1b: You can view the array representation of the Series:

```
obj1.values
```

```
array([ 4,  7, -1,  2])
```

And view the index object:

```
obj1.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

Example 1c: It's often desirable to create a Series with an index that labels each data point:

```
obj2 = pd.Series([4, 7, -1, 2], index = ['x', 'y', 'z', 't'])
```

```
x    4
y    7
z   -1
t    2
dtype: int64
```

Example 1d: You can index a Series using the array index or the array label (see notebook):

```
obj2[2]
-1

obj2['z']
-1
```

Slicing Series

Example 2: Let's look at the Series we created:

```
obj2 = pd.Series([4, 7, -1, 2], index = ['x', 'y', 'z', 't'])
```

Let's suppose we slice it:

```
obj2[0:2]

x    4
y    7
dtype: int64
```

What about?

```
obj2['x':'z']

x    4
y    7
z   -1
dtype: int64
```

Example 3a: Beware! **Implicit Python slicing** versus **explicit pandas indexing**
Here's where some confusion comes in:

```
data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])

1    a
3    b
5    c
dtype: object


data[1]
'a'

data[1:3]

3    b
5    c
dtype: object
```

Example 3b: Pandas provides indexer attributes to reference the explicit index:

```
data.loc[1]
'a'

data.loc[1:3]

1    a
3    b
dtype: object
```

Example 3c: On the other hand, `iloc` uses implicit indexes:

```
data.iloc[1]
'b'

data.iloc[1:3]

3    b
5    c
dtype: object
```

(Note: The `ix` attribute was introduced as a hybrid but has not been included since pandas 2.0)

Example 4a: You can perform NumPy-like functions and operations, including masking:

```
obj2[obj2 > 2]

x    4
y    7
dtype: int64
```

Example 4b: Since a Series operates as an ordered dictionary, you can use operations that we used previously on dicts:

```
't' in obj2
True
```

Example 5a: You can create a Series from a dict:

```
canines = {"Spot": 9.2, "Lassie":24.5, "Fido": 102.1}
dogs = pd.Series(canines)
dogs

Spot        9.2
Lassie     24.5
Fido      102.1
dtype: float64
```

Notice the data alignment features.

Example 5b: The Series object and its index have a name attribute that can be set:

```
dogs.name = 'weight'
dogs.index.name = 'petname'
dogs
```

```
petname
Spot         9.2
Lassie      24.5
Fido       102.1
Name: weight, dtype: float64
```

DataFrames

DataFrame – a rectangular table of data, an ordered collection of columns of various types, has both a row and a column index, a dictionary of aligned Series (share the same index) but stored as 1- and 2-dimensional array blocks.

Example 6a: For example, let's define a dict and use it to create a DataFrame.

```
canines2 = {'petname': ['Spot', 'Spot', 'Spot', 'Lassie', 'Lassie', 'Lassie', 'Fido', 'Fido', 'Fido'],
            'months': [4, 10, 18, 4, 8, 16, 3, 8, 18],
            'weight' : [2.1, 4.5, 9.2, 6.0, 15.9, 24.5, 10.2, 64.1, 102.1]}
df = pd.DataFrame(canines2)
```

Example 6b: Use `df.head()` to view the first 5 rows (or `df.tail()` to view the last 5 rows)

	petname	months	weight
0	Spot	4	2.1
1	Spot	10	4.5
2	Spot	18	9.2
3	Lassie	4	6.0
4	Lassie	8	15.9
5	Lassie	16	24.5
6	Fido	3	10.2
7	Fido	8	64.1
8	Fido	18	102.1

Example 6c: You can retrieve a column as a Series either by dict notation or by attribute:

`df['petname']` or `df.petname`

```
0      Spot
1      Spot
2      Spot
3    Lassie
4    Lassie
```

```
5    Lassie
6      Fido
7      Fido
8      Fido
Name: petname, dtype: object
```

Example 6d: Add a column as an indication of being up-to-date on heartworm meds:

```
df['heartworm'] = True
```

	petname	months	weight	heartworm
0	Spot	4	2.1	True
1	Spot	10	4.5	True
2	Spot	18	9.2	True
3	Lassie	4	6.0	True
4	Lassie	8	15.9	True
5	Lassie	16	24.5	True
6	Fido	3	10.2	True
7	Fido	8	64.1	True
8	Fido	18	102.1	True

Example 6e: Suppose Spot and Fido need a 2nd year rabies vaccine and no information is available for Lassie.

```
rabies = pd.Series(["True", "True", "False", "True", "True", "False"], index =[0,1,2,6,7,8])
df["vac"] = rabies
```

	petname	months	weight	heartworm	vac
0	Spot	4	2.1	True	True
1	Spot	10	4.5	True	True
2	Spot	18	9.2	True	False
3	Lassie	4	6.0	True	NaN
4	Lassie	8	15.9	True	NaN
5	Lassie	16	24.5	True	NaN
6	Fido	3	10.2	True	True
7	Fido	8	64.1	True	True
8	Fido	18	102.1	True	False

Example 6f: Suppose we add a column and then decide to delete it.

```
df['fleas'] = False
df.columns
```

```
Index(['petname', 'months', 'weight', 'heartworm', 'vac', 'fleas'], dtype='object')
```

```
del df['fleas']
df.columns
```

```
Index(['petname', 'months', 'weight', 'heartworm', 'vac'], dtype='object')
```

Example 7: We can create a DataFrame from a 2D ndarry with optional row and column labels:

```
import numpy as np
newdf = pd.DataFrame(np.random.randn(3,2), columns = ['yes', 'no'],
                                    index = ['yesterday', 'today', 'tomorrow'])
```

	yes	no
yesterday	0.065132	0.501395
today	-0.464964	-1.250192
tomorrow	-1.176962	-1.027148

Subsetting/Slicing DataFrames

Example 8a: We can subset by columns using a single or multiple column names:

```
df[['petname', 'vac', 'heartworm']]
```

	petname	vac	heartworm
0	Spot	True	True
1	Spot	True	True
2	Spot	False	True
3	Lassie	NaN	True
4	Lassie	NaN	True
5	Lassie	NaN	True
6	Fido	True	True
7	Fido	True	True
8	Fido	False	True

Example 8b: We can subset by rows using a single or multiple row index label (here default labels):

`df.loc[2]` or `df.iloc[2]` Why the same?

```
petname       Spot
months          18
weight         9.2
heartworm     True
vac          False
Name: 2, dtype: object
```

`df.loc[[2,4]]` or `df.iloc[[2,4]]`

	petname	months	weight	heartworm	vac
2	Spot	18	9.2	True	False
4	Lassie	8	15.9	True	NaN

Example 8c: We can subset a row based upon a Boolean expression:

```
df[df['months'] >= 18]
```

	petname	months	weight	heartworm	vac
2	Spot	18	9.2	True	False
8	Fido	18	102.1	True	False

Example 8d: We can subset rows based on slicing notation:

```
df[2:4]
```

	petname	months	weight	heartworm	vac
2	Spot	18	9.2	True	False
3	Lassie	4	6.0	True	NaN

Example 8e: We can subset based upon a list

```
df[df['petname'].isin(["Spot","Fido"])]
```

	petname	months	weight	heartworm	vac
0	Spot	4	2.1	True	True
1	Spot	10	4.5	True	True
2	Spot	18	9.2	True	False
6	Fido	3	10.2	True	True
7	Fido	8	64.1	True	True
8	Fido	18	102.1	True	False

Index Objects

Series and DataFrame objects contain an index object to reference and assist in modifying the data.

Example 9a: To construct an Index object:

```
ind = pd.Index([2, 3, 5, 7, 11])
```

```
Index([2, 3, 5, 7, 11], dtype='int64')
```

Example 9b: The Index object has many of the same attributes as NumPy arrays:

```
ind.size, ind.shape, ind.ndim, ind.dtype
```

```
5, (5,), 1, dtype('int64')
```

Example 9c: However, an Index object is immutable, so the following results in an error! Solution: share indices between DataFrames and arrays.

```
ind[1] = 100
```